

PRACTICAL – 01

Name: Gunjan Chute

PRN:- 202501050025

Division: Chemical(CH2)

1.1.1 Calculate Momentum

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula: where: is the mass of the object (in kilograms). is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Code:

```
mass = float(input()) velocity
```

```
= float(input())
```

```
momentum = mass*velocity print(f"{momentum:.2f}kgm/s")
```

The screenshot displays the CodeTANTRA IDE interface. At the top, the header includes the CodeTANTRA logo, navigation links (Home, Learn Anywhere), a user profile (202501050025@mitaoe.ac.in), and a Logout button. The main workspace shows the Explorer panel on the left and the Debugger panel on the right. The central area displays the test results for the program. Two test cases are shown, both of which passed. Test case 1 had an average time of 0.014 s and a maximum time of 0.015 s. Test case 2 had an average time of 0.013 s and a maximum time of 0.015 s. The expected output for both test cases is 50.00kgm/s and 11.04kgm/s, respectively. The actual output for both test cases is 50.00kgm/s and 11.04kgm/s, respectively. The bottom of the interface features a navigation bar with buttons for Prev, Reset, Submit, and Next.

Test Case	Average time	Maximum time	Status	Expected output	Actual output
Test case 1	0.014 s	0.015 s	2 out of 2 shown test case(s) passed	50.00kgm/s	50.00kgm/s
Test case 2	0.013 s	0.015 s	2 out of 2 hidden test case(s) passed	11.04kgm/s	11.04kgm/s

1.1.2. Conditional Calculation Based on the Number of Digits

Write a Python program that accepts an integer as input. Depending on the number of digits in .

Constraints:

$$1 \leq n \leq 999$$

Input Format:

- The input consists of a single integer .

Output Format:

- If n is a single-digit number, print its square.
- If n is a two-digit number, print its square root (rounded to two decimal places).
- If n is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".

Code:

```
n=int(input()) if
0<=n and n<=9:
    print(n*n)      elif
10<=n and n<=99:
    print(f"{n**(1/2):.2f}") elif
100<=n and n<=999:
    print(f"{n**(1/3):.2f}") else:
    print("Invalid")
```

CodeTANTRA Home Learn Anywhere 202501050025@mitaoe.ac.in Support Logout

Average time: 0.007 s, Maximum time: 0.010 s, 4 out of 4 shown test case(s) passed, 3 out of 3 hidden test case(s) passed

Test case	Expected output	Actual output
Test case 1	9	9
Test case 2	166	166
Test case 3	24	24
Test case 4	3456	3456

Prev Reset Submit Next

1.1.3. Days between Two Dates

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format YYYY-MM-DD
- The second line contains the second date in the format YYYY-MM-DD
- An integer representing the number of days between the given dates.

Output Format:

Code: from datetime import datetime

```
date1=input().strip() date2=input().strip()
```

```
d1=datetime.strptime(date1,"%Y-%m-%d")
```

```
d2=datetime.strptime(date2,"%Y-%m-%d")
```

```
difference = d2-d1 print(difference.days)
```

The screenshot displays the CODETANTRA online IDE interface. At the top, the header includes the logo, navigation links (Home, Learn Anywhere), a user email (202501050025@mitaoe.ac.in), and a Logout button. The main workspace shows the execution results of a Python program. On the left, a sidebar contains an Explorer and a Debugger. The central area displays performance metrics: Average time (0.018 s, 17.75 ms) and Maximum time (0.039 s, 39.00 ms). It also shows test case results: '2 out of 2 shown test case(s) passed' and '2 out of 2 hidden test case(s) passed'. Two test cases are detailed below. Test case 1, with a duration of 39 ms, shows an expected output of '2014-07-02', '2014-11-02', and '123', which matches the actual output. Test case 2, with a duration of 10 ms, shows an expected output of '2014-07-02', '2014-07-11', and '9', which also matches the actual output. At the bottom, a navigation bar includes buttons for '< Prev', 'Reset', 'Submit', and 'Next >'.

1.1.4. Reverse a Number

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Code:

```
num = int(input()) reversed_num=
```

```
int(str(num)[::-1])
```

```
print(reversed_num)
```

[Home](#)
[Learn Anywhere](#)

202501050025@mitaoe.ac.in
[Support](#)
[Logout](#)

Average time

0.006 s

5.60 ms

Maximum time

0.007 s

7.00 ms

2 out of 2 shown test case(s) passed

3 out of 3 hidden test case(s) passed

Test case 1

7 ms

Debug

Expected output	Actual output
5367	5367
7635	7635

Test case 2

4 ms

Debug

Expected output	Actual output
0132	0132
231	231

Terminal

Test cases

< Prev

Reset

Submit

Next >

1.1.5. Multiplication Table

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

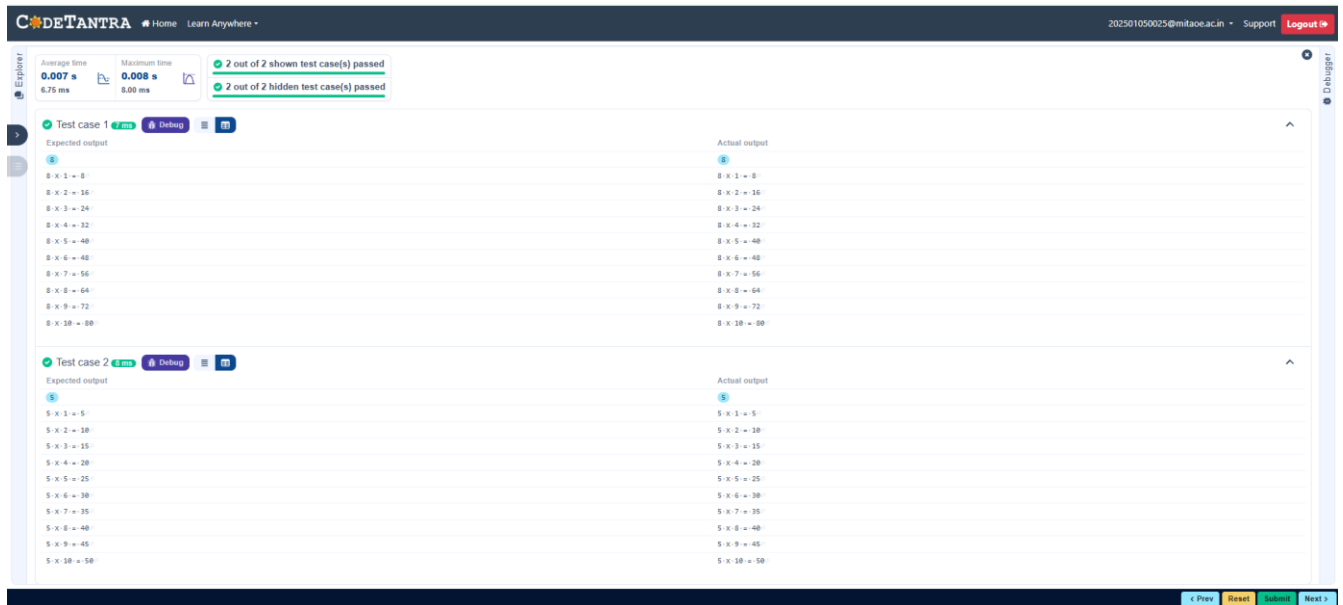
- Print the multiplication table for the given number in the format: **<number> x**

<multiplier> = <result> Code:

```
num = int(input()) for
```

```
i in range(1,11):
```

```
    print(f"{num} x {i} = {num * i}")
```



1.2.1. Pass or Fail

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer , the number of courses.
- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Code:

```

def cal(marks, total_courses):
    if any(mark < 40 for mark in marks):
        return "Fail"

    total_marks = sum(marks)

    aggregate_percentage=(total_marks/(total_courses *100))*100

    if aggregate_percentage >75:
        grade="Distinction"
    elif aggregate_percentage >=60:
        grade= "First Division"
    elif aggregate_percentage >=50:
        grade= "Second Division"
    elif aggregate_percentage >=40:
        grade= "Third Division"

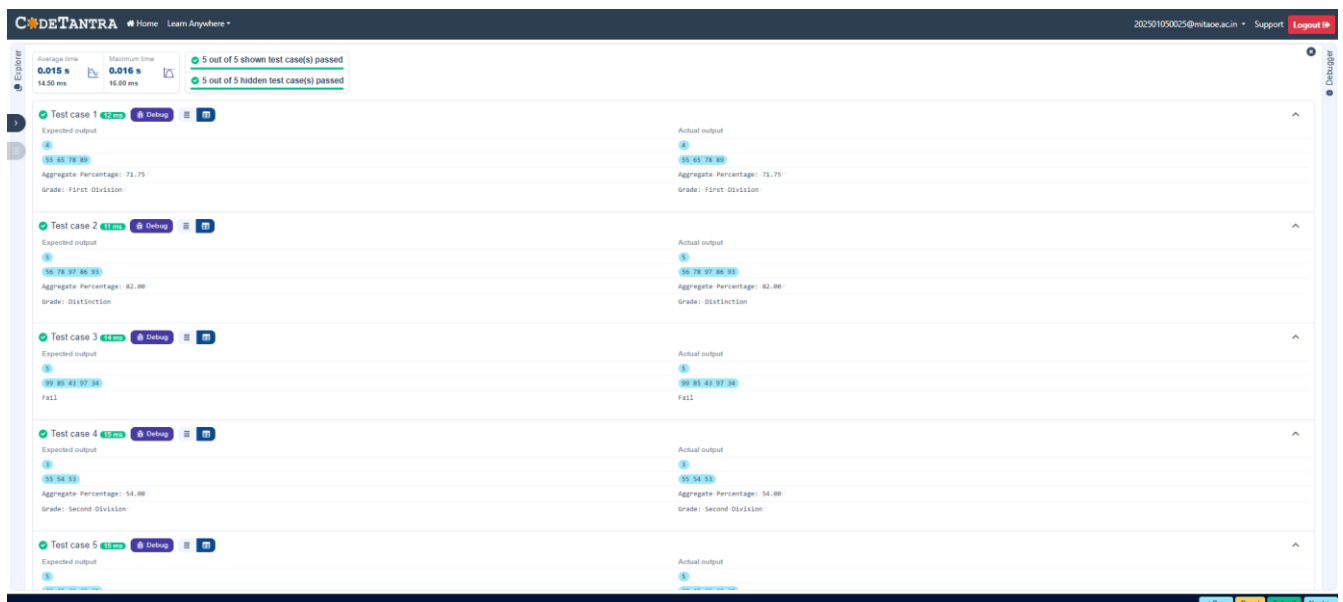
    return (aggregate_percentage,grade)

num_courses= int(input())
marks= list(map(int,input().split()))

result= cal(marks,num_courses)

if result == "Fail":
    print("Fail")
else:
    aggregate_percentage,grade=result
    print(f"Aggregate Percentage: {aggregate_percentage:.2f}")
    print(f"Grade: {grade}")

```



1.2.2. Fibonacci Series

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

- A single integer representing the number of terms to generate.

Output Format:

- A single line containing the first terms of the Fibonacci sequence, separated by spaces.

Code: def

fibonacci(n):

if n == 1:

return 0

elif n == 2:

return 1

else:

return fibonacci(n - 1) + fibonacci(n - 2)

n = int(input())

for i in range(1, n + 1):

print(fibonacci(i), end=" ")

The screenshot shows the CODETANTRA online code editor interface. At the top, there's a navigation bar with the logo, home link, and user information. The main area displays the execution results of a Python program. It shows two test cases, both of which passed. The first test case had an expected output of '5' and an actual output of '5'. The second test case had an expected output of '15' and an actual output of '15'. The interface also includes a sidebar with 'Explorer' and 'Debugger' tabs, and a bottom bar with navigation buttons like 'Prev', 'Reset', 'Submit', and 'Next'.

1.2.3. Pattern -1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern. **Output Format**
- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Code:

```
n = int(input()) for i in
range (1, n+1):
print('* ' * i)
```

The screenshot shows the CODETANTRA online IDE interface. At the top, there's a navigation bar with 'Home' and 'Learn Anywhere' links, a user email '202501050025@mitaoe.ac.in', and a 'Logout' button. The main area displays the execution results of a Python program. It shows 'Average time 0.007 s' and 'Maximum time 0.012 s'. A green banner indicates '2 out of 2 shown test case(s) passed' and '4 out of 4 hidden test case(s) passed'. Below this, there are sections for 'Expected output' and 'Actual output', both showing a right-angled triangle pattern of numbers. The 'Test case 2' section shows a 'Debug' button and a 'Terminal' tab. At the bottom, there are buttons for '< Prev', 'Reset', 'Submit', and 'Next >'.

1.2.4. Pattern -2

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Code:

```
n = int(input()) for i in
range(1, n+1): for j in
range(1, i+1):
print(j,end=" ")
print()
```


CODETANTRA

HomeLearn Anywhere

202501050025@mitaoe.ac.inSupportLogout

Explorer

Average time0.007 s7.50 ms

Maximum time0.008 s8.00 ms

2 out of 2 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 18 msDebug

Expected output

5

1-1

1-2

1-2-3

1-2-3-4

1-2-3-4-5

Actual output

5

1-1

1-2

1-2-3

1-2-3-4

1-2-3-4-5

Test case 28 msDebug

Expected output

8

1-1

1-2

1-2-3

1-2-3-4

1-2-3-4-5

1-2-3-4-5-6

1-2-3-4-5-6-7

1-2-3-4-5-6-7-8

Actual output

8

1-1

1-2

1-2-3

1-2-3-4

1-2-3-4-5

1-2-3-4-5-6

1-2-3-4-5-6-7

1-2-3-4-5-6-7-8

PrevResetSubmitNext